

METRICMIND

Enterprise Analytics & Agentic AI

PROJECT 1 — FINAL PROJECT REPORT

Organization	Axlero Solutions
Project	MetricMind — Enterprise Analytics & Agentic AI
Team Lead	Shyam
Date	August 2026
Repository	https://github.com/ShyamVarmora/MetricMind/tree/devops

Submission-ready documentation covering the business problem, architecture, implementation, conversational analytics, governance, testing posture and requirement alignment.

1. Executive Summary

MetricMind is an enterprise analytics application that lets users ask business questions in natural language and receive data-backed answers through a controlled backend query layer. The application combines dashboard analytics, reporting, conversational analysis, authentication and configurable UI behavior in one workspace.

The source Project 1 brief identifies unrestricted text-to-SQL as a trust problem: incorrect joins and inconsistent metric definitions can make finance and sales disagree about the same business number.

Submission principle: this report separates source requirements from repository implementation. The repository uses React + Vite, FastAPI/Python and MySQL, while the source brief names Next.js/Tremor, Cube.dev/dbt and Snowflake/Databricks. The difference is documented explicitly rather than hidden.

Key outcomes

- Natural-language business querying through Chat.
- Support for conversational turns such as greetings and help requests.
- Sales/revenue, cost, shipping, profit, margin, orders, customers, products, quantity and average-order-value metrics.
- Geography, gender, date, year and quarter filtering and grouped analysis.
- Multi-step European margin analysis comparing quarters and drilling into product cost and freight.
- Dynamic line/bar visualization for structured chat results.
- Reasoning steps, API trace, SQL and parameters for data-backed turns.

2. Business Problem & Objectives

Business teams need fast answers to questions such as “Q3 Revenue”, “Show me European sales”, “Why did sales go up?” and “Why did our European margins drop last quarter?”. Dashboards answer predefined questions well, but become less flexible when users need to explore a new metric, dimension or time period.

The project therefore treats governance as a product requirement: the user speaks in business language, the application maps that request to controlled metrics and filters, the backend queries the data and the system returns an explanation that can be inspected.

Objectives

- Business-friendly analytics interface.
- Natural-language questions without unrestricted SQL access.
- Deterministic metric and filter handling.
- Root-cause analysis rather than only a single KPI.
- Visible data/query path for auditability.
- Bounded database exploration.
- Readable light and dark themes.

Acceptance checks from the source brief

Requirement	Expected behavior
Governance audit	Repeated “Q3 Revenue” should return the same numerical result without SQL hallucination.
API check	“Show me European sales” should map to a governed data request.
Multi-step reasoning	Margin drop should trigger a secondary shipping/material cost breakdown.
Dynamic visualization	Line chart for time series; bar chart for categorical results.
Cost governance	Query limits prevent unbounded expensive exploration.
Transparency	Expose View API Call / View SQL where applicable.

3. Architecture & Technology

The implemented repository follows a practical web application architecture. The frontend is React + Vite. The backend is FastAPI/Python and exposes REST endpoints for authentication, analytics and conversational business queries. MySQL stores the sales fact table and supporting dimensions.

Logical flow

USER	FRONTEND	BACKEND	DATABASE
Natural-language question	React + Vite UI	FastAPI + governed query agent	MySQL fact + dimension tables

Implemented application areas

- Dashboard: executive KPI cards, sales trend and recent transactions.
- Reports: reporting-oriented UI and outputs.
- Analytics: date/category filters, KPI cards, trends and breakdowns.
- Chat: natural-language analysis, reasoning path, visual result, data table and query transparency.
- Profile: authenticated user profile.
- Settings: application preference and theme controls.

Technology alignment note: the source brief names Next.js + Tremor, Cube.dev/dbt and Snowflake/Databricks. The current repository substitutes React + Vite, FastAPI and MySQL. This report does not claim the source components are present when they are not.

4. Data Model & Business Metrics

The current backend works against the Internet Sales-style schema represented by factinternetsales and dimension tables. The business-query layer centralizes the metric expressions used by the assistant.

Metric	Definition
Sales / Revenue	SUM(SalesAmount)
Cost	SUM(TotalProductCost) + SUM(Freight)
Shipping Cost	SUM(Freight)
Profit	SalesAmount – TotalProductCost – Freight
Margin %	Profit / SalesAmount × 100
Orders	COUNT(DISTINCT SalesOrderNumber)
Customers	COUNT(DISTINCT CustomerKey)
Products	COUNT(DISTINCT ProductKey)
Quantity	SUM(OrderQuantity)
Average Order Value	SalesAmount / distinct orders

Supported analysis dimensions include geography, customer, product, gender, occupation, color, product line, marital status, education, income, month, quarter and year.

Centralizing metric definitions is important because the same business metric should not be silently redefined by each user question.

5. Conversational AI Workflow

The Chat workflow classifies the user's request. Conversational requests such as “Hello” are handled without a database query. Business questions are parsed into a controlled metric, filters and optional grouping. The backend executes a parameterized query and formats the result as a business answer.

Example workflows

Question	Workflow
Hello	Conversational response; no database query required.
Q3 Revenue	Revenue metric + quarter filter → controlled query → answer.
Show me European sales	Europe filter → sales result → business explanation.
Why did sales go up?	Monthly trend → month-over-month comparison → measured increase.
Why did our European margins drop last quarter?	European quarter comparison → margin change → product-cost/shipping breakdown → root-cause explanation.

Multi-step root-cause path

- Filter the dataset to the European geography scope.
- Compare the latest two available quarters.
- Calculate the margin percentage change.
- Compare product-cost and shipping-cost movement.
- Explain the largest measured driver without inventing unavailable fields.

6. Dynamic Visualization & User Experience

Structured chat results can be rendered visually. Time-based groupings such as month, quarter and year use a line chart, while categorical results such as product or country use a bar chart. A structured table remains available so values are still inspectable.

The Analytics area adds date-range and category filtering and displays KPI cards, a time series, top-product visualization and a category breakdown table.

Theme correction

Shared CSS theme variables control page background, surfaces, text, muted text, borders, primary color and shadows. Chat cards, tables, forms, error states and analytics surfaces are explicitly styled for dark mode.

UX behavior

- Visible loading and retry states.
- Backend error details can be surfaced rather than leaving a blank screen.
- Chat history is persisted locally and can be cleared.
- Question drafts are retained locally when appropriate.
- Responsive layout rules support smaller screens.

7. Governance, Security & Transparency

Governance is central to the project brief. The implemented agent rejects direct SQL/database commands from user input and maps supported questions into predefined business metrics and filters.

Control	Implementation / posture
No user SQL	Direct SQL/database commands are rejected.
Parameterized queries	User-derived filters are supplied as parameters.
Bounded execution	Governance records query operations and applies limits.
Traceability	Reasoning steps, API trace, SQL and parameters are returned where applicable.
Authentication	Protected routes use bearer-token authentication.

The source documentation also describes a semantic-layer-only target with strict JSON and a five-query / 1,000-row governance model. The current repository implements the governance idea at the FastAPI agent/database layer rather than through Cube.dev.

8. Testing & Quality Evidence

The evidence pack contains historical runtime/API testing and screenshots. Earlier evidence recorded 14 API checks, with 8 passing and 6 failing; Dashboard, Reports and Analytics were documented historical high-priority defects. Those results predate the latest code changes and are not presented as a fresh current-run result.

Current submission posture

- Final repository branch: devops.
- Latest code changes address the observed AI timeout, conversational handling, multi-step margin analysis, analytics error handling and theme visibility.
- The current code was reviewed at source level for this package; this document does not claim that a fresh end-to-end runtime test was executed inside the document-generation environment.
- A final live demonstration should verify the acceptance questions against the current branch.

Recommended final acceptance script

#	Test	Expected
1	Login → Dashboard	Verify protected access and KPI rendering.
2	Analytics filters	Change date/category filters and confirm reload.
3	Hello	Confirm conversational response without database query.
4	Q3 Revenue	Repeat twice and compare numerical result.
5	European sales	Confirm Europe filtering and structured result.
6	Why did sales go up?	Confirm monthly trend and measured increase.
7	European margin question	Confirm multi-step cost breakdown and reasoning.
8	Dark theme	Verify cards, tables, text and inputs remain readable.

9. Requirement Compliance Matrix

The matrix uses evidence-based wording. “Implemented” refers to functionality represented in the current repository; “Equivalent” means the behavior is achieved through a different technical component than the source stack.

Requirement	Source expectation	Current submission
Governed business querying	Controlled metrics/filters	Implemented in current FastAPI agent layer
Conversational BI	Natural-language questions	Implemented in Chat
Multi-step reasoning	Secondary cost-driver analysis	Implemented
Dynamic visualization	Line/bar by result type	Implemented
Cost governance	Bounded query execution	Governance layer present
Transparency	View API / SQL trace	Implemented
Dashboard & analytics	KPIs, filters, trends	Implemented
Authentication	Protected application areas	Implemented
Theme / UX	Readable light/dark modes	Implemented

Stack difference: the source brief is explicit about Cube.dev/dbt, Snowflake/Databricks and Next.js/Tremor. The current repository is React + Vite, FastAPI/Python and MySQL. The report intentionally separates those facts.

10. Team & Execution

The project documentation records the following responsibilities.

Member	Responsibility
Shyam	Team Lead • DevOps • Integration
Chetan	Backend • Agent Orchestration
Mansi	Data • Database • Semantic Validation
Yoshita	UI • Frontend
Himesh	Frontend • Reassigned Work
Prashant	QA • API Testing • Documentation

Four-phase execution model

- Week 1 — Warehouse/modeling foundation and chat scaffolding.
- Week 2 — Semantic definitions and agent prompt/orchestration logic.
- Week 3 — Governance audit, API checks and multi-step reasoning.
- Week 4 — Dynamic visualization, cost governance, transparency and final polish.

GitHub is the project source of truth for the current submission branch.

11. Limitations & Review Notes

A strong technical submission distinguishes what is implemented from what is specified. The following points are intentionally explicit.

- The source architecture and repository technology stack are not identical.
- The repository uses MySQL rather than the source brief's Snowflake/Databricks target.
- The application uses React + Vite rather than Next.js + Tremor.
- The governed query behavior is implemented in FastAPI/Python rather than through Cube.dev.
- Historical test counts should not be treated as a fresh final-run result.
- The final live demonstration should verify the acceptance questions against the current branch.

This is the correct engineering posture: a credible report documents actual implementation and verification status rather than claiming an unverified stack or runtime result.

12. Final Conclusion & Submission Checklist

MetricMind provides a coherent enterprise analytics application with conversational business querying, structured analytics, governance controls, dynamic visualization and transparent reasoning. The implementation is documented as an actual repository deliverable rather than a theoretical architecture.

Final submission checklist

- Project repository: MetricMind / devops branch.
- Final PowerPoint presentation.
- Final PDF project report.
- README and repository documentation retained in GitHub.
- Final live acceptance test covering Q3 Revenue, European sales, sales trend, European margin root cause and light/dark theme visibility.

Repository

<https://github.com/ShyamVarmora/MetricMind/tree/devops>

Prepared for Project 1 final review • MetricMind • August 2026

13. Source Basis

This report is based on the project materials available in the workspace and the current MetricMind repository.

- Axlero Solutions — Project Document: Advanced Data Analytics, Project 1: “MetricMind”: Agentic Semantic BI Engine.
- Axlero Solutions — Standard Operating Procedures for Intern Projects, Version 1.1.
- MetricMind architecture and implementation documentation.
- MetricMind repository: GitHub branch devops.
- Existing project evidence report, screenshots and testing documentation.

The source project explicitly requires governance, consistent metrics, API checks, multi-step reasoning, dynamic visualization, cost governance and transparency. The human-readable source list above provides provenance for the report.

END OF REPORT